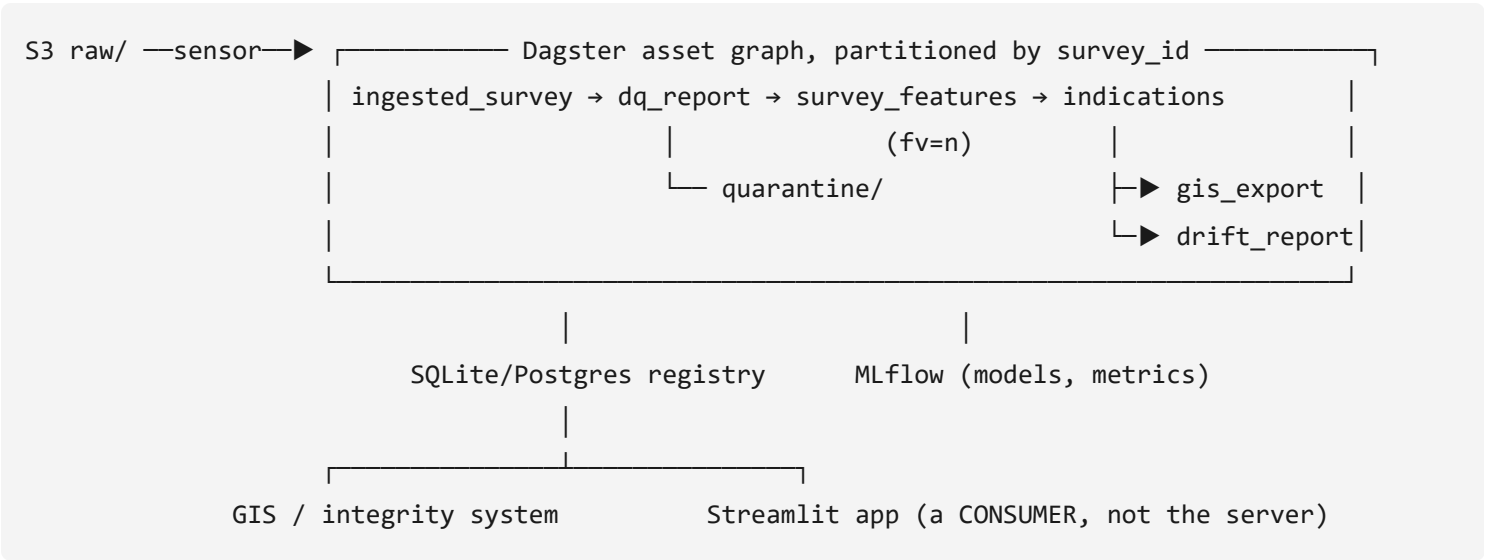


Production architecture — LSM integrity pipeline

How this would run at ROSEN scale: thousands of surveys, years of archive, ground truth that arrives months late. The demonstrator builds a subset of this (§10); the rest is designed, costed and written down rather than hand-waved.

Design centre: the primary workload is **backfill over an existing archive**, not steady-state scoring. Everything below follows from that.

1. System at a glance



Dagster's software-defined assets map 1:1 onto this: each box is an asset, `survey_id` is a dynamic partition, `feature_version` is the asset's code version, and **backfill is a first-class partition backfill** rather than a bespoke script. That fit is why Dagster over Airflow here.

In plain English:

- **"Dagster asset graph"** — the diagram above (raw survey → cleaned survey → data-quality check → features → predictions → reports) is a flowchart of steps where each step's output feeds the next. Dagster is just the tool that runs this flowchart automatically instead of a person manually running each script in order. Dagster's word for "one box in the flowchart" is an **asset** — it's not mysterious, it's literally "the thing this step produces" (e.g. `dq_report` is the asset that the data-quality step produces).
- **"Partitioned by `survey_id`"** — instead of treating the whole archive as one giant job that has to run start-to-finish in one go, Dagster slices the work into one slice per survey (`survey_id` is just the unique name of one survey, like a filename). Each slice can run, fail, retry, or be re-run completely on its own. If survey #4,187 out of 6,000 fails, you only have to fix and re-run #4,187 — the other 5,999 are untouched.
- **"dq_report"** — "dq" = data quality. This is just the small report/record produced by the data-quality-check step: for one survey, it lists which automated checks passed, which warned, and which failed. It is **not a piece of software or a "tech stack"** — it's plain output data (think: a table or a JSON file), similar to a spell-checker's list of flagged words. If a survey fails its checks, it gets set aside ("quarantined") instead of being used to make predictions.
- **"Backfill is a first-class partition backfill rather than a bespoke script"** — "backfill" means running the pipeline over *old* surveys that were never processed (or need reprocessing), not just today's newest upload. Because Dagster already treats "one survey" as an independent, retryable unit (the partition described above), asking it to backfill 6,000 old surveys is a built-in feature: Dagster tracks which of the 6,000 have finished, retries the failures, lets you pause and resume, and limits how many run at once. The alternative — "a bespoke script" — would mean someone hand-writing a one-off Python loop over the old surveys with no automatic retry, no resume-after-crash, and no built-in progress tracking; all of that would have to be built and debugged separately. Using Dagster's backfill feature means we get all of that for free instead of reinventing it.

2. Environments and config layering

Three environments, three separate buckets / databases / MLflow experiments. Three config layers with different lifecycles — conflating them breaks reproducibility:

In plain English — what's a "bucket", and what actually differs between dev and prod?

A **bucket** just means an **S3 bucket** — Amazon's cloud storage, basically a folder / hard drive that lives on the internet instead of your laptop. "Three separate buckets" means dev, staging, and prod each get their own completely separate storage folder for raw surveys, model files, etc., so a test run in dev can never accidentally read or overwrite prod's real data.

Looking at the actual config files (`config/dev.yaml` vs `config/prod.yaml`) in this repo:

	dev	prod
Database	local SQLite file on disk	same (SQLite here too — prod is a placeholder, see below)
Cloud storage (S3)	off — everything stays on your machine	on — files go to a real S3 bucket
MLflow (experiment tracker)	points at a local file	points at a real tracking server
Parallel workers	4 at once	16 at once

Important honesty note: this project never actually runs against real cloud infrastructure — `prod.yaml` is a placeholder that documents the *intended* shape (see §10, "designed and documented only"). It exists to prove the environment-separation mechanism works, not because there's a real production deployment.

Layer	Contents	Lifecycle	Hashed into the run?
Code config <code>config/base.yaml</code>	detrend degree, windows, quantiles, gate thresholds	git commit	yes — this is <code>config_sha256</code>
Environment config <code>config/{dev,staging,prod}.yaml</code>	bucket, DB URI, MLflow URI, concurrency	per environment	no
Runtime params	which partition, which <code>pipeline_version</code>	per invocation	logged, not hashed

Only the code layer feeds `config_sha256` . Otherwise the same code and data produce a different reproducibility hash in prod than in dev, which silently voids the guarantee.

In plain English — what would go wrong if we didn't do this?

`config_sha256` is a fingerprint (a hash) of the settings that can affect *what the model predicts* — things like how the background trend is removed, window sizes, thresholds. The idea is: "same fingerprint = same code path, so results should be reproducible."

The environment settings (bucket name, database file path, MLflow server address, how many things run in parallel) do **not** change what the model predicts — they only change *where things are stored* and *how fast it runs*. If those were mixed into the same fingerprint, then running the exact same analysis code on the exact same data in dev vs. prod would produce two *different* fingerprints, even though the actual scientific result is identical. That would make the fingerprint useless for proving "this result is reproducible" — hence "only the code layer feeds `config_sha256`."

Runtime parity is enforced, not recorded. Training (`ml_gpu` , Python 3.12.13), the local venv (3.13) and the app runtime are three dependency graphs today. Production pins one Python version and one lockfile, training runs in a container referenced **by digest** in the bundle, and `bundle.load()` **hard-fails** on a LightGBM/numpy/sklearn version mismatch rather than merely recording it. A silent minor-version difference changes predictions with no error — the worst class of failure.

In plain English — has this actually been fixed, or is it just a plan?

Yes, it's already implemented in code, not just described here. In `project/src/lsm/bundle.py` , the function `load_bundle()` (around lines 112–143): every time a saved model is loaded, the code compares the exact numpy / scikit-learn / LightGBM version numbers that were installed when the model was *trained* against whatever versions are installed *right now*. If any of them differ even slightly, it raises an error (`BundleVersionMismatchError`) and **refuses to load the model at all** — it doesn't just print a warning and carry on.

Why this matters: libraries like scikit-learn occasionally change their internal math in small, undocumented ways between versions (e.g. a slightly different random-number algorithm, or a bugfix that shifts a calculation by a tiny amount). If that happened silently, the model would keep running and keep producing predictions — just very slightly different ones — and nothing would tell you. That's what "the worst class of failure" means: not a crash (which you'd notice), but a quiet, undetected change in the answers. Making it hard-fail instead of just logging turns that invisible risk into a loud, obvious error that stops the run.

3. Orchestration, triggering and backfill

Concern	Production
Trigger	S3 sensor on <code>raw/</code> for new surveys; nightly schedule as a safety net
Unit of work	one survey = one partition; idempotent and independently retryable
Retries	3× exponential backoff on transport/IO; zero on DQ failure — a bad survey is not a flake
Backfill	Dagster partition backfill with bounded concurrency; resumable — a failure at survey 4,000 of 6,000 costs one partition, not the run
Concurrency	feature computation fans out across cores; registry writes are serialised (SQLite is single-writer; Postgres above $\sim 10^7$ rows or any multi-writer ingest)
Isolation	one poison survey quarantines itself and the run continues

In plain English — the whole table, one row at a time:

"3× exponential backoff on transport/IO; zero on DQ failure — a bad survey is not a flake." —

"Transport" here just means *moving the raw file from one place to another* — e.g. downloading a survey file from cloud storage (S3) or reading it off disk. That step can fail for boring, temporary reasons that have nothing to do with the data itself: a network hiccup, the storage service being briefly overloaded, a timeout. Those are worth retrying — "exponential backoff" just means "wait a bit, try again; if it fails again, wait longer, try again" (this codebase does it 3 times, in `src/lsm/dagster_defs.py`). A **"flake"** is the technical slang for exactly this kind of failure — one that goes away if you simply try again, like a Wi-Fi drop-out. But if the file downloaded *fine* and the data-quality check then finds something genuinely wrong with the survey (bad readings, corrupted values), that is not a flake — trying again 3 times won't fix bad data, it's the same file every time. So the pipeline retries the "moving the file" step, but never retries "the data was bad."

"Dagster partition backfill with bounded concurrency; resumable — a failure at survey 4,000 of 6,000 costs one partition, not the run." —

Recall from §1 that each survey is its own independent slice ("partition"), like one checkbox on a 6,000-item to-do list. Reprocessing the whole archive means ticking off all 6,000 checkboxes. **"Bounded concurrency"** means the computer doesn't try to do all 6,000 at the exact same instant — it works on a limited number at a time (so it doesn't run out of memory or overload the database), moving to the next one as each finishes. **"Resumable"** means Dagster keeps track of which checkboxes are already ticked. So if survey #4,000 crashes the process, you don't have to start over from #1 — you fix #4,000 and pick back up from there. Only that one survey's work is lost ("costs one partition"), not the other 5,999 that already succeeded ("not the run").

How idempotent and quarantine actually work, concretely (not just as buzzwords): - Idempotent

means "running the same step twice on the same survey gives the same result, and doesn't create duplicates or corrupt anything." In `src/lsm/ingest.py`, `register_survey()` checks the database first: if this exact survey is already registered, it just logs "already have this one" and does nothing (returns `"noop"`) instead of inserting a second copy. That's what makes it safe for Dagster to re-run a survey that already partly succeeded — it won't double-count it. - **Quarantine** means: when the data-quality check finds a serious problem with a survey, the code (in `src/lsm/validate.py`) does two concrete things — (1) it flips that survey's status in the database to `"quarantined"` so nothing downstream will use it, and (2) it copies the raw file into a separate `data/quarantine/` folder for a human to look at later. Nothing is deleted; it's set aside, like putting a damaged package in a "return to sender" bin instead of shipping it out.

4. Serving contract — batch, and distinct from the demo app

The serving layer is the **batch scoring job**. The Streamlit app is a *consumer* of its output. Conflating the two is a category error.

Trigger	asset materialisation on new/updated <code>survey_features</code>
Output	<code>indication</code> rows + <code>reports/<survey_id>/indications.geojson</code> (versioned schema)
Consumer	GIS / integrity management system
Contract	output schema is versioned and covered by a consumer contract test in CI; a breaking change requires a new schema version, not an edit
Idempotency	re-scoring the same <code>(survey_id, pipeline_version)</code> replaces its rows exactly
Latency SLO	indications available < 24 h after survey upload; < 30 s compute per 20 km survey

In plain English:

"Asset materialisation on new/updated `survey_features`" — "materialisation" is just Dagster's word for "this step actually ran and produced/saved its output" (as opposed to just being planned). `survey_features` is the specific step/box from the §1 diagram that turns a cleaned survey into the numeric features the model reads (background-removed signal, window statistics, etc.) for **one particular survey**. So this row just says: the prediction step (`predict`) is not on a timer — it automatically kicks off *the moment* the features step finishes producing (or re-producing) features for a survey, rather than someone having to manually trigger scoring afterward.

"Output schema is versioned and covered by a consumer contract test in CI; a breaking change requires a new schema version, not an edit." — The `indications.geojson` file (the file that gets handed to the GIS/integrity system) has an agreed-upon shape: which fields it contains, their names, their types — like a form template both sides agreed on. That template is given a version number (`schema_version` , see the release manifest in §5). "Covered by a consumer contract test in CI" means there's an automated test that runs on every code change and checks the actual output still matches that template — if someone renamed or removed a field, the test would fail and stop that change from shipping. The rule "a breaking change requires a new schema version, not an edit" means: if the shape genuinely needs to change, you don't quietly reshape the existing file (which would break whatever the GIS system expects to see) — you publish a *new* version number so the consumer can knowingly upgrade to it on their own schedule, the same way an API v1/v2 would work. *(Honesty check on this demonstrator: there is a test — `test_predict.py` — that verifies the GeoJSON output has the right shape; a full formal "block the change unless the version bumps" gate against a real downstream GIS system is one of the items §10 lists as designed/costed rather than fully built, since there's no real external consumer to test against here.)*

5. Release and rollback

The deployable unit is a `pipeline_version` **release manifest**, not an individual model — an indication is produced by up to four models plus a feature version, and all of them must be pinned together for its lineage to be answerable.

```
pipeline_version: 2026.07.29-a1b2c3d
models: {anomaly: anom-..., severity: sev-..., classify: cls-..., growth: grw-...}
feature_version: 3
schema_version: 2
container_digest: sha256:...
```

- **MLflow 3 removed model stages.** Promotion sets the `@champion alias`; `@challenger` marks the shadow candidate. Aliases make promotion and rollback atomic pointer moves with no state mutation.
- **Rollback = repoint `@champion` to the previous `pipeline_version`.** Bundles are immutable, so a rollback target always exists.
- **Rollback is drilled in CI,** because the usual way it fails is that the old bundle no longer loads against current code — which our own `feature_version` guard would cause. Policy: the last **N = 3** feature versions stay loadable, and CI proves it.
- **Re-scoring policy on promotion:** historical surveys are *not* automatically re-scored; every analytical query filters on `pipeline_version`. Re-scoring the archive is a deliberate, scheduled backfill. Without this rule, growth forecasting silently compares outputs from different models.

In plain English — this whole section, piece by piece:

"The deployable unit is a `pipeline_version` release manifest, not an individual model." — One prediction ("indication") on a survey doesn't come from a single model — it comes from up to four separate models working together (one that spots something unusual, one that rates how severe it is, one that guesses the defect type, one that forecasts how it'll grow), plus a specific version of the feature-computing code. If you upgraded just one of those five pieces on its own, you couldn't tell anymore *which combination* actually produced a given prediction — so instead of shipping/upgrading models one at a time, they're always bundled and released together as one package, tagged with a single version number (`pipeline_version` , e.g. `2026.07.29-a1b2c3d`). Think of it like a phone's OS update: you don't update the camera app and the keyboard app separately and hope they still work together — you install one combined "build."

The YAML block above is just what's literally written down for one such release: which exact model was used for each of the four jobs, which version of the feature code, which version of the output format, and a fingerprint (`container_digest`) of the exact software environment it ran in. It's a receipt, not code that runs.

"MLflow 3 removed model stages... promotion sets the `@champion` alias." — MLflow is just the tool used to keep track of trained models. Older versions of that tool let you label a model as "Staging" or "Production" ; that feature was removed. The replacement is simpler: `@champion` is just a name tag/sticky-note that always points at "whichever model is currently live," and `@challenger` points at "the one being tried out side-by-side, not yet trusted." Promoting a new model just means moving that `@champion` sticky note to point at it — nothing about the old or new model itself changes.

"Rollback = repoint `@champion` to the previous `pipeline_version` . Bundles are immutable." — "Rollback" = undo a bad release and go back to the one before it. Because nothing about a past release is ever edited or deleted after the fact ("immutable" — once saved, a bundle is frozen forever), rolling back is as simple as moving that same `@champion` sticky note back to where it used to point. There's no risk of "the old version isn't there anymore" because old versions are never thrown away.

"Rollback is drilled in CI... the last N = 3 feature versions stay loadable." — "Drilled in CI" means: this isn't just a plan on paper, there's an automated test that actually *performs* a rollback and checks it works, every time the code changes (CI = "continuous integration," the automatic test-runner). The reason this needs testing at all: a rollback can quietly fail if the *old* model was built using an older version of the feature-computing code, and the *current* code refuses to load anything that old (recall from earlier answers: `bundle.py` intentionally hard-fails on version mismatches). So the team made a policy — keep the last 3 versions of that feature code working — and proved with a test that rolling back to any of those 3 actually still loads successfully.

"Re-scoring policy on promotion: historical surveys are not automatically re-scored." — When a better model is promoted, old surveys that were already scored by the *previous* model are **left alone** — nobody goes back and reruns them through the new model automatically. Every stored prediction remembers which `pipeline_version` made it, and anyone querying the results has to say which

version's results they want. Re-running the whole historical archive through a new model is treated as its own deliberate, planned job (a "backfill," same concept as earlier), not something that happens silently in the background. The reason this matters: if old and new predictions got silently mixed together, something like "is this defect growing?" (which compares a prediction from years ago to a recent one) could actually be comparing two different models' opinions and mistake that difference for real physical growth.

6. Delayed labels — the operating model

Ground truth arrives months late and is **censored by our own past ranking**: we only dig where we predicted. This is the defining constraint of the domain and it drives three mechanisms.

1. **Vintage backtesting.** Every evaluation pins `truth_as_of` and reconstructs what was known on that date, then scores forward. Reported metrics are always as-of a stated date.
2. **Shadow scoring.** Champion and challenger both score every live survey. Since outcome comparison must wait months, the near-term signal is **disagreement analysis**: change in indication count per km, rank churn at the dig budget, and whether disagreements concentrate in DQ-warned surveys. This is the only pre-label validation available, and it works from day one.
3. **Feedback loop and exploration.** Each excavation writes a `truth_defect` revision. Because labels are censored, a small fraction of the dig budget should be allocated by *uncertainty* × *consequence* rather than by rank — otherwise the model can only ever confirm itself. Tracked metric: **median days from indication to verification**, which bounds how fast anything can be learned.

7. Monitoring, alerting, runbook

Model observability: feature PSI/KS vs the bundle's training reference (>0.2 warn,

0.3 block), indications-per-km vs training rate, background regime shift per line, measured conformal coverage as verifications arrive.

In plain English — how do we actually detect and report drift?

"Drift" means: the new surveys coming in today look statistically different from the surveys the model was originally trained on — which is a warning sign the model might not perform as well as it did during training. It's produced by running `python -m lsm monitor` (code in `src/lsm/monitor.py`), which compares a new survey's data against a saved snapshot of what the training data looked like, using two standard statistics tests:

- **PSI (Population Stability Index)** — a single number that measures how much a feature's distribution (e.g. the typical signal strength) has shifted compared to training. Bigger number = bigger shift. The actual cutoffs used in this codebase: above **0.2** → **"warn"**, above **0.3** → **"block"** (stop scoring until a human looks at it).
- **KS (Kolmogorov-Smirnov test)** — a second, independent statistical comparison of two distributions, used as a cross-check alongside PSI.

On top of those two, the drift check also looks at: how many defects-per-km the model is flagging now vs. during training (a sudden jump could mean something's off), and whether the raw background magnetic-field readings on a given survey line look like that same line's past surveys ("background regime shift" — e.g. a scanner was swapped or recalibrated). Only the PSI "block" threshold actually halts scoring automatically; the other checks raise a flag for an analyst to review, they don't stop the pipeline by themselves.

System observability: structured JSON logs correlated by `survey_id` / `run_id` / `pipeline_version` ; per-asset duration, rows/sec, failure count, quarantine rate. Alerts route to the integrity-analytics on-call channel — an alarm with no destination is not monitoring.

Symptom	First check	Action
Survey quarantined	<code>dq_report</code> check name	Analyst review; re-acquire if sensor fault
Quarantine rate > 5%	Is it one line or one scanner?	Suspect acquisition, not data
PSI > 0.3	Background regime vs that line's history	Block scoring; usually a new/recalibrated scanner
Bundle load fails	<code>feature_version</code> / library digest	Blocked deploy — rebuild or roll back
Coverage below nominal	Recent verification residuals	Refit conformal quantiles, do not retrain

8. Evaluation discipline

- Grouped splits by `(line_id, block)` ; spatial **and** temporal holdouts.

- **Point-in-time correctness:** every feature for a survey must be computable from data available at that survey's `surveyed_at` . Enforced by an as-of join and a test — spatial grouping does not catch temporal leakage.
- **Final test set is touched rarely and its use is counted.** Repeatedly promoting against a fixed holdout selects on holdout noise; the release process itself is subject to multiple comparisons. Rotate as new lines arrive.
- All headline metrics reported as bootstrap CIs over groups; gates compare intervals.

9. SLOs, retention, cost

SLO	Target
Survey ingest + validate	< 2 min
Indications available	< 24 h after upload
Backfill throughput	≥ 500 surveys/hour
Quarantine rate	< 5% (alert above)
Scheduled GIS export delivered	99% of runs

Retention: raw surveys indefinitely (immutable — they *are* the asset; Glacier IR after 12 months); features 90 days then recompute on demand (they're derivable); quarantine 12 months; indications indefinitely (audit trail); MLflow artifacts — champion versions forever, others pruned at 12 months.

Cost: ~500 surveys/year at 20 km and 0.5 m spacing is $\sim 2 \times 10^7$ rows/year, well under 1 GB/year compressed.
Storage is not the cost. The cost is compute during archive backfill and analyst time reviewing indications — which is exactly why *false-dig rate*, not accuracy, is the metric the system optimises.

10. Built here vs designed here

Built in the demonstrator	Designed and documented only
Dagster asset graph, retries, one real backfill	Airflow/K8s or AWS Batch at full scale
Config layering, env separation (dev/prod)	Staging environment with its own infrastructure
<code>pipeline_version</code> manifest, MLflow aliases, rollback drill	Blue/green infrastructure
Shadow scoring + disagreement report	Multi-year vintage backtest
Point-in-time test, bundle version assertion	Full consumer contract with a real GIS system
Structured logging, drift report	Paging, on-call rotation, dashboards

The distinction is deliberate: a demonstrator that tries to build all of this ships nothing. What matters is that every box on the right has a named decision behind it, not a gap.